

Example workflow files (will share)

1: upload your image to the “example_project” folder.

2: change image_filename in “sgt_to_rgt_files.py” to the name of the file you uploaded

3: adjust sgt settings in sgt_to_rgt_files.py

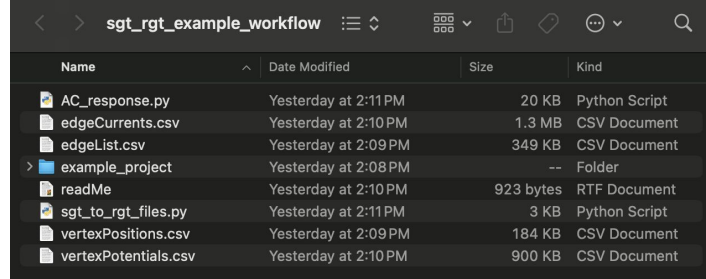
* I don't know what the best settings for a given image are

4: run sgt_to_rgt_files.py

5: change the imposed potential and physical parameters in “AC_response.py” to whatever you your imposed potential is

6: run AC_response.py

7: the vertex potentials and edge currents will be exported as csv files



Name	Date Modified	Size	Kind
AC_response.py	Yesterday at 2:11 PM	20 KB	Python Script
edgeCurrents.csv	Yesterday at 2:10 PM	1.3 MB	CSV Document
edgeList.csv	Yesterday at 2:09 PM	349 KB	CSV Document
> example_project	Yesterday at 2:08 PM	--	Folder
readMe	Yesterday at 2:10 PM	923 bytes	RTF Document
sgt_to_rgt_files.py	Yesterday at 2:11 PM	3 KB	Python Script
vertexPositions.csv	Yesterday at 2:09 PM	184 KB	CSV Document
vertexPotentials.csv	Yesterday at 2:10 PM	900 KB	CSV Document

SGT can easily give the adjacency matrix, but it isn't very useful for RGT

```
# You can get the adjacency matrix if you need it|
A = skel_system.graph.get_adjacency()
print(f"Graph generated. Adjacency matrix shape: {A.shape}")
```

RGT needs the list of **vertex** positions and **edges** (source, target)

```
N=skel_system.Gr.vcount()

verts=skel_system.Gr.vs
vertpos = np.array([verts[n]['o'] for n in range(N)])

eds = skel_system.Gr.es
Ne = len(eds)
edgelist = np.array([[eds[n].source,eds[n].target] for n in range(Ne)])

df = pd.DataFrame(vertpos)

# Export the DataFrame to a CSV file without header and index
df.to_csv('vertexPositions.csv', header=False, index=False)

df = pd.DataFrame(edgelist)

# Export the DataFrame to a CSV file without header and index
df.to_csv('edgeList.csv', header=False, index=False)
```

RGT takes these files as inputs

```
edges = pd.read_csv('edgeList.csv', header=None, index_col=False).to_numpy()
```

```
vertpos = pd.read_csv('vertexPositions.csv', header=None, index_col=False).to_numpy()
```

You need to give RGT the applied potential that you want

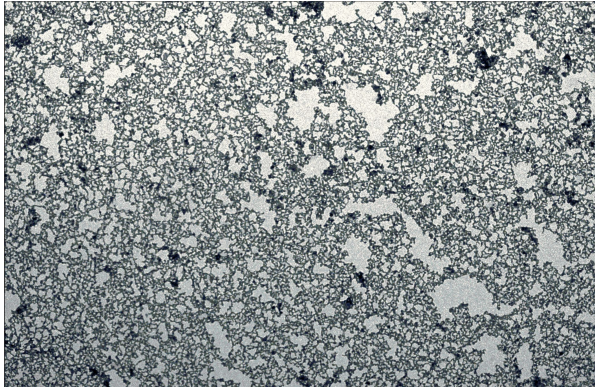
```
##making the imposed potential -- THIS IS WHAT YOU NEED TO CHANGE TO GET DIFFERENT RESPONSES ON THE SAME SYSTEM, here we just apply a top-bottom potential
given_potential_fraction = 0.05
given_potential_magnitude = 100
num_selected = int(given_potential_fraction * n_vertices)
given_potential_list = np.zeros(int(2*num_selected)) #Ultimately this is what gets passed on, the other code in this block just makes this a top-bottom potential
vertex_list = np.zeros(int(2*num_selected)) #Ultimately this is what gets passed on, the other code in this block just makes this a top-bottom potential
# Sort vertices by y-position
sorted_vertices = sorted(enumerate(vertpos), key=lambda x: x[-1][-1]) # Sort by y-coordinate
# Select the top and bottom vertices
top_vertices = sorted_vertices[-num_selected:] # Top f% (highest y-values)
bottom_vertices = sorted_vertices[:num_selected] # Bottom f% (lowest y-values)
top_list = [idx for idx, _ in top_vertices]
bottom_list = [idx for idx, _ in bottom_vertices]
for i in range(len(top_list)):
    given_potential_list[i] = given_potential_magnitude
    vertex_list[i] = top_list[i]
for i in range(len(bottom_list)):
    given_potential_list[len(top_list)+i] = -given_potential_magnitude
    vertex_list[len(top_list)+i] = bottom_list[i]

#example parameters, set to very large/small numbers for DC response
given_potential_frequency = 0.000000000001
resistivity=1
capacitance=0.000000000001
inductance=0.000000000000000000000001
leak_resistivity=1000000000

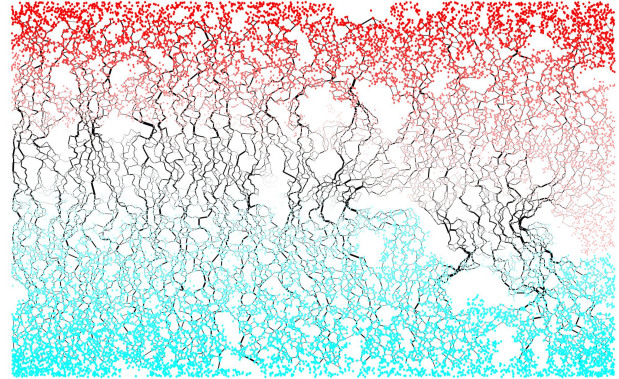
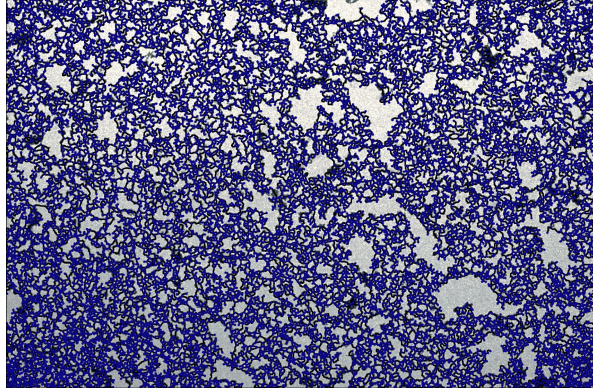
rlist = resistivity*np.ones(len(edges))
llist = inductance*np.ones(len(edges))
clist = capacitance*np.ones(n_vertices-2*num_selected)
rllist = leak_resistivity*np.ones(n_vertices-2*num_selected)
```

These are the outputs of the two scripts:

sgt_to_rgt_files.py:



AC_response.py



My biggest remaining question: how do I know what the best processing parameters are?

I can usually play around with options to get a good network, unless the image has some gradient of visual quality.

```
# --- 2. Define Image Processing Options ---
processing_options = {
    "Thresh_method": 0,
    "thresh": 128.0,
    "gamma": 1.001,
    "md_filter": 0,
    "g_blur": 0,
    "autolvl": 0,
    "fg_color": 1, #1 for black network, 0 for white network
    "laplacian": 0,
    "scharr": 0,
    "sobel": 0,
    "lowpass": 0,
    "asize": 3,
    "bsize": 11,
    "wsize": 1,
}
```

